

Sohum Pohane

Lab 2

```
In [1]: from charlm import train_char_lm, print_probs, generate_text, normalize
        from collections import *
        from math import *
```

a)

```
In [2]: mylm = train_char_lm('subtitles.txt',4)
        print_probs(mylm, 'atio')
```

```
[('n', 0.9940436161014506),
 (' ', 0.00220962628494572),
 ('.', 0.0013930252665962147),
 (',', 0.0009607070804111826),
 ('?', 0.0003362474781439139),
 ('"', 0.00024017677010279565),
 ('u', 0.00019214141608223654),
 (''', 0.0001441060620616774),
 ('s', 0.0001441060620616774),
 ('-', 9.607070804111827e-05),
 ('!', 4.8035354020559135e-05),
 (':', 4.8035354020559135e-05),
 ('m', 4.8035354020559135e-05),
 ('p', 4.8035354020559135e-05),
 ('r', 4.8035354020559135e-05)]
```

```
In [3]: print_probs(mylm, 'nivi')
```

```
[('n', 0.8), ('e', 0.1), ('s', 0.1)]
```

```
In [4]: print_probs(mylm, 'supe')
```

```
[('r', 0.9992144540455616), ('s', 0.0007855459544383347)]
```

```
In [5]: for i in range(5):
        print("-----")
        print(generate_text(mylm,4,80))
```

Favority mum.
 – Davis, but."
 Where.
 Statemen, would go crows and I thing a fathe

A report.
 It's just happing clearlie?
 Exercised to king to be the on.
 What have

– You have for about kill me anotheriffs is course.
 Louis Brad.
 Pyramifinance of

Tac Taylor.
 In factually do who weekend.
 I living to seeing to report of down fo

Are you!
 – Okay, I'm going? "
 YOU BE GOT NOW THANK YOU.
 – Right not.
 Mr. Grace a

b)

```
In [6]: def perplexity(text, lm, order=4):
# Pad the input with "~" chars. This handles the case where order > len
pad = "~" * order
data = pad + text
# This is a stub.
# Loop over data string and find probs and use to compute perplexity
#return float("inf")
logProbSum = 0.0
charCount = 0

for i in range(len(data) - order):
    history = data[i:i+order]
    char = data[i+order]

    if history in lm:
        found_prob= False
        for c, prob in lm[history]:
            if c ==char:
                if prob > 0:
                    logProbSum+= log2(prob)
                    charCount+= 1
                    found_prob= True
```

```

        break
    if not found_prob:
        return float("inf")
    else:
        return float("inf")
    if charCount == 0:
        return float("inf")
    crossEntropy = -(logProbSum/charCount)
    result=2**crossEntropy

    return result

```

```
# end of file
```

```
In [7]: perplexity('The boy loves his mother', mylm, 4)
```

```
Out[7]: 3.9091903673746224
```

```
In [8]: perplexity('The student loves homework', mylm, 4)
```

```
Out[8]: 4.606972940490915
```

```
In [9]: perplexity('The yob loves homework', mylm, 4)
```

```
Out[9]: inf
```

```
In [10]: perplexity('It is raining in London', mylm, 4)
```

```
Out[10]: 3.711236000904451
```

```
In [11]: perplexity('asdfjkl; qwerty', mylm, 4)
```

```
Out[11]: inf
```

c)

```

In [12]: # Computes per-char perplexity of a text, given an input LM. Smoothing is v
def smoothed_perplexity(text, lm, order=4):
    # Pad the input with "~" chars. This handles the case where order > len
    pad = "~" * order
    data = pad + text
    # This is a stub.
    # Loop over data string and find probs and use to compute perplexity
    logProbSum=0.
    charCount=0
    smoothing_constant =1.0e-7

```

```

for i in range(len(data) - order):
    history= data[i:i+order]
    char =data[i+order]
    prob =smoothing_constant
    if history in lm:
        for c, p in lm[history]:
            if c ==char:
                prob= p
                break
        logProbSum+= log2(prob)
        charCount+= 1
    if charCount == 0:
        return float("inf")
    crossEntropy=-(logProbSum/charCount)
    result= 2**crossEntropy

return result

```

In [13]: `smoothed_perplexity('The boy loves his mother', mylm, 4)`

Out[13]: 3.9091903673746224

In [14]: `smoothed_perplexity('The student loves homework', mylm, 4)`

Out[14]: 4.606972940490915

In [15]: `smoothed_perplexity('The yob loves homework', mylm, 4)`

Out[15]: 71.98159797830623

In [16]: `smoothed_perplexity('It is raining in London', mylm, 4)`

Out[16]: 3.711236000904451

In [17]: `smoothed_perplexity('asdfjkl; qwerty', mylm, 4)`

Out[17]: 2344635.5209805984

d)

```

In [18]: def predict(text, models, order=0):
          best = None
          bestPerplexity = float('inf')
          perplexities = {}

          for lang_code, model in models.items():
              perp = smoothed_perplexity(text, model, order)

```

```

        perplexities[lang_code] = perp
        if perp < bestPerplexity:
            bestPerplexity = perp
            best = lang_code
    return best, perplexities

```

```

In [19]: def train_language_identification_models(order):
    languages = {
        'en': 'en.train.txt',
        'da': 'da.train.txt',
        'de': 'de.train.txt',
        'fr': 'fr.train.txt',
        'it': 'it.train.txt',
        'nl': 'nl.train.txt'
    }
    models = {}

    for lang_code, filename in languages.items():
        models[lang_code] = train_char_lm(filename, order=order)
    return models

```

```

In [20]: def evaluate_language_identification(filename, order=0):

    models = train_language_identification_models(order)

    with open(filename, 'r', encoding='utf-8') as f:
        totalPredictions = 0
        totalCorrect = 0
        stats = {}

        for line_num, line in enumerate(f, 1):
            line = line.strip()
            if not line:
                continue

            parts = line.split('\t')
            trueLang, sentence = parts

            predictedLang, perplexities = predict(sentence, models, order=or

            if line_num == 1:
                sortedPerplexities = sorted(perplexities.items(), key=lambda
                print("First test sentence:")
                print(sortedPerplexities)
            is_correct = (predictedLang == trueLang)
            totalPredictions += 1
            if trueLang in stats:
                stats[trueLang]['total'] += 1
            else:

```

```

        stats[trueLang]={'correct':0,'total':1}

    if is_correct:
        totalCorrect += 1
        stats[trueLang]['correct'] += 1

    for lang in sorted(stats.keys()):
        stat = stats[lang]
        accuracy = stat['correct']/stat['total'] if stat['total'] > 0 else 0
        print(f"{lang}: {stat['correct']} correct out of {stat['total']}")

```

In [21]: `evaluate_language_identification('test.txt',0)`

First test sentence:

```
[('fr', 21.23036140889083), ('it', 23.15318659107709), ('nl', 26.3193826897344), ('da', 28.990266275714863), ('de', 29.17749118527597), ('en', 31.052440781065986)]
```

```
da: 192 correct out of 200 lines - 96.0%
de: 184 correct out of 200 lines - 92.0%
en: 191 correct out of 200 lines - 95.5%
fr: 194 correct out of 200 lines - 97.0%
it: 196 correct out of 200 lines - 98.0%
nl: 181 correct out of 200 lines - 90.5%
```

In [22]: `evaluate_language_identification('test.txt',1)`

First test sentence:

```
[('fr', 10.569891704413088), ('nl', 22.476777267969254), ('de', 27.675256068736), ('it', 27.716461785294275), ('da', 27.79772641085918), ('en', 32.191582354972205)]
```

```
da: 199 correct out of 200 lines - 99.5%
de: 198 correct out of 200 lines - 99.0%
en: 200 correct out of 200 lines - 100.0%
fr: 196 correct out of 200 lines - 98.0%
it: 200 correct out of 200 lines - 100.0%
nl: 198 correct out of 200 lines - 99.0%
```

In [23]: `evaluate_language_identification('test.txt',3)`

First test sentence:

```
[('fr', 6.431649027658283), ('nl', 128.26246243331676), ('en', 212.82666013776205), ('it', 324.3407861725636), ('de', 540.0699921420306), ('da', 1062.0441991047046)]
```

```
da: 200 correct out of 200 lines - 100.0%
de: 199 correct out of 200 lines - 99.5%
en: 200 correct out of 200 lines - 100.0%
fr: 199 correct out of 200 lines - 99.5%
it: 200 correct out of 200 lines - 100.0%
nl: 199 correct out of 200 lines - 99.5%
```

As we see the accuracy gets better when we change the model from a unigram model to

a bigram model, and then even better when we try a 4-gram model. We can tell this by looking at the improved accuracies of the language identification.

e)

```
In [24]: def train_tennis_models(filename,order=0):
    gender = {'M': [], 'F': []}
    models = {}

    with open(filename, 'r', encoding='utf-8') as f:
        totalPredictions = 0
        totalCorrect = 0
        stats = {}

        for line_num, line in enumerate(f, 1):
            line = line.strip()
            if not line:
                continue

            parts = line.split('\t')
            gend, sentence = parts
            gender[gend].append(sentence)

        for key in gender.keys():
            sents = gender[key]
            lm = defaultdict(Counter)
            for s in sents:
                pad = "~" * order
                data = pad + s + '\n'
                for i in range(len(data)-order):
                    history, char = data[i:i+order], data[i+order]
                    lm[history][char] += 1
            outlm = {hist:normalize(chars) for hist, chars in lm.items()}
            models[key] = outlm
    return models
```

```
In [25]: def evaluate_tennis(filename,order=0):

    models = train_tennis_models('tennis.train.txt',order)

    with open(filename, 'r', encoding='utf-8') as f:
        totalPredictions = 0
        totalCorrect = 0

        stats = {}

        for line_num, line in enumerate(f, 1):
```

```

line = line.strip()
if not line:
    continue

parts = line.split('\t')
trueGend, sentence = parts
predictedGend, perplexities = predict(sentence, models, order=ord
if line_num == 1:
    print("First test sentence:")
    sortedPerplexities = sorted(perplexities.items(), key=lambda
    print(sortedPerplexities)
is_correct = (predictedGend == trueGend)
totalPredictions += 1
if trueGend in stats:
    stats[trueGend]['total'] += 1
else:
    stats[trueGend]={'correct':0,'total':1}

if is_correct:
    totalCorrect += 1
    stats[trueGend]['correct'] += 1
for gend in sorted(stats.keys()):
    stat = stats[gend]
    accuracy = stat['correct']/stat['total'] if stat['total']>0 else
    print(f"{gend}: {stat['correct']} correct out of {stat['total']}")

```

In [26]: `evaluate_tennis('tennis.test.txt',0)`

First test sentence:
 [('M', 20.040028857920287), ('F', 20.043442327551922)]
 F: 1945 correct out of 3696 lines – 52.6%
 M: 2665 correct out of 4518 lines – 59.0%

In [27]: `evaluate_tennis('tennis.test.txt',1)`

First test sentence:
 [('F', 10.52245125509176), ('M', 10.600984745459169)]
 F: 2449 correct out of 3696 lines – 66.3%
 M: 2542 correct out of 4518 lines – 56.3%

In [28]: `evaluate_tennis('tennis.test.txt',2)`

First test sentence:
 [('F', 6.411370651765233), ('M', 6.48324412876136)]
 F: 2658 correct out of 3696 lines – 71.9%
 M: 2697 correct out of 4518 lines – 59.7%

In [29]: `evaluate_tennis('tennis.test.txt',3)`

First test sentence:

```
[('F', 5.0482437541900715), ('M', 5.1482664446371365)]
```

F: 2471 correct out of 3696 lines – 66.9%

M: 2935 correct out of 4518 lines – 65.0%

In [30]: `evaluate_tennis('tennis.test.txt',4)`

First test sentence:

```
[('M', 5.495399570011996), ('F', 6.23768820627894)]
```

F: 2296 correct out of 3696 lines – 62.1%

M: 2945 correct out of 4518 lines – 65.2%

It looks like the 4-gram model works the best in terms of having good accuracies for both genders. Even though the 3-gram model gives a higher accuracy for F, it gives a low accuracy comparatively for M.

In []: